國立陽明交通大學

NATIONAL YANG MING CHIAO TUNG UNIVERSITY

Artificial Intelligence

Assignment I

Parking Occupancy Detection Based on a Self-Collected Dataset:A Comparative Study of HOG+SVM, YOLO26, and Single-Class YOLO Approaches

指導教授：王才沛博士

姓名：劉　順　松

學號：414551016

基於自建資料集之停車場車位在席辨識：HOG+SVM、YOLO26 與單類別 YOLO 方法之比較研究

Parking Occupancy Detection Based on a Self-Collected Dataset:A Comparative Study of HOG+SVM, YOLO26, and Single-Class YOLO Approaches

GitHub：https://github.com/414551016/AI-Capstone/tree/main/AI-Hw1

Abstract

本研究探討停車場車位在席辨識（Parking Occupancy Detection）問題，並以「有車（occupied）」與「無車（empty）」作為二元分類任務，以及僅使用單類別（occupied）進行訓練之 YOLO 模型。依據作業要求，本研究自行蒐集停車場影像，建立自建資料集，並使用 Roboflow 對每個車位進行標註，再透過 Python 程式將標註框裁切為單一車位小圖，以利後續模型訓練與比較。作業規定資料集不得直接使用現成資料，且需自行建構資料、進行 supervised learning、比較至少兩種方法，並以 cross-validation 進行評估。

本研究選用兩種 supervised learning 方法：其一為傳統機器學習方法 HOG + SVM，其二為深度學習方法 YOLO26。前者以固定大小之車位小圖為輸入，萃取 Histogram of Oriented Gradients 特徵後，使用 Support Vector Machine 進行分類；後者則基於 YOLO26 進行物件偵測，以輸出車位之 occupied 與 empty 狀態。根據實驗結果，HOG + SVM 在目前資料集上取得極高之分類表現，而 YOLO26 在驗證集上亦獲得優異之 Precision、Recall 與 mAP。惟進一步分析後發現，固定攝影機與相似背景可能造成資料分割過於樂觀，因此本研究亦針對資料洩漏（data leakage）與泛化能力進行討論。

本研究結果顯示，在固定場景下，傳統方法與深度學習方法皆可有效完成停車位在席辨識；然而就實務應用而言，YOLO26 對於複雜情境與未來擴充性具有較高潛力。最後，本文亦提出研究限制與未來工作方向，包括夜間影像、不同停車場、跨格停車與更嚴謹之 group-based validation 設計。

實驗結果顯示，HOG+SVM 在固定場景下可達到 100% 準確率，而 YOLO26 在驗證集上取得 mAP@0.5 = 0.994 與 mAP@0.5:0.95 = 0.911 的優異表現。此外，單類別 YOLO 模型在降低標註成本的同時仍能達到良好辨識效果。進一步分析指出，資料場景固定與資料分割方式可能導致過度樂觀結果，因此本研究亦針對資料洩漏與模型泛化能力進行討論。

關鍵字：

Introduction

1.1 Research Background：

在智慧停車管理系統中，車位是否被占用是最重要的資訊之一。若能利用影像分析即時判斷停車場中各車位的 occupied/empty 狀態，便能有效提升停車效率、降低尋位時間，並可進一步應用於智慧停車導引、停車場管理與交通資訊整合等場景。

相較於使用感測器、地磁或超音波之方式，影像式停車位辨識具有布建成本較低、資訊量較高與可視化程度較強之優點。但影像方法也面臨光照、陰影、背景固定、遮蔽、車

身顏色與停車姿態差異等問題，因此如何建立合適資料集並比較不同學習方法，為本研究重點。

1.2 Motivation：

本作業的核心精神不只是「跑模型」，而是完整經歷資料建構、模型訓練、實驗設計與結果分析的流程。作業明確要求：資料集需自行建立、不可直接使用既有 dataset，且需至少比較兩種 supervised learning 方法，並以 cross-validation 評估模型表現。

傳統方法如 HOG + SVM 可有效進行影像分類，但在複雜環境下表現有限；而深度學習方法（如 YOLO）在物件偵測任務中展現出更強的特徵學習能力。因此，本研究比較兩種方法在停車場場景下的效能差異。

因此，本研究選擇停車場車位在席辨識作為主題，理由如下：

1. 標籤清楚：有車 / 無車
2. 可自行拍攝並建立資料集
3. 可設計成二元分類問題
4. 適合比較傳統機器學習與深度學習方法
5. 可延伸分析資料量、背景相似性與資料洩漏問題

1.3 Research Objective：

本研究的目標如下：

1. 將每個車位標註為 occupied 或 empty。
2. 比較 HOG + SVM 與 YOLO26 兩種 supervised learning 方法。
3. 以 cross-validation 與驗證集指標分析模型表現。
4. 探討資料場景固定對模型評估的影響。

1.4 Research Question：

本研究主要回答下列問題：

1. 在自建資料集上，HOG + SVM 是否能有效辨識停車位之 occupied / empty 狀態？
2. YOLO26 在停車位偵測與分類任務上的表現如何？
3. 傳統機器學習與深度學習方法在此問題上的差異為何？
4. 固定攝影機與相似背景是否造成過度樂觀的結果？

Problem Definition

本研究將停車場車位在席辨識定義為一個二元分類問題。對於每個車位區域，模型需判斷其狀態為：

- occupied：車位被車輛占用
- empty：車位未被車輛占用

在資料處理階段，本研究另保留以整張影像進行偵測的版本，以利 YOLO26 使用；但在 HOG＋SVM 中，主要輸入為已裁切之單一車位小圖。需注意的是，若採用「總車位數 - 偵

測車輛數」的方式推估空位數，當出現一車跨兩格或停車壓線時，容易產生誤差。因此本研究主體採用「以車位為單位」之 occupied/empty 判斷，可較自然處理一車占用兩格之情況。

Dataset Construction

2.1 Data Collection：

本研究資料由學生在新竹市晶品城購物廣場 4F － 11F 停車場自行拍攝，符合「不可使用現成 dataset」之規定。

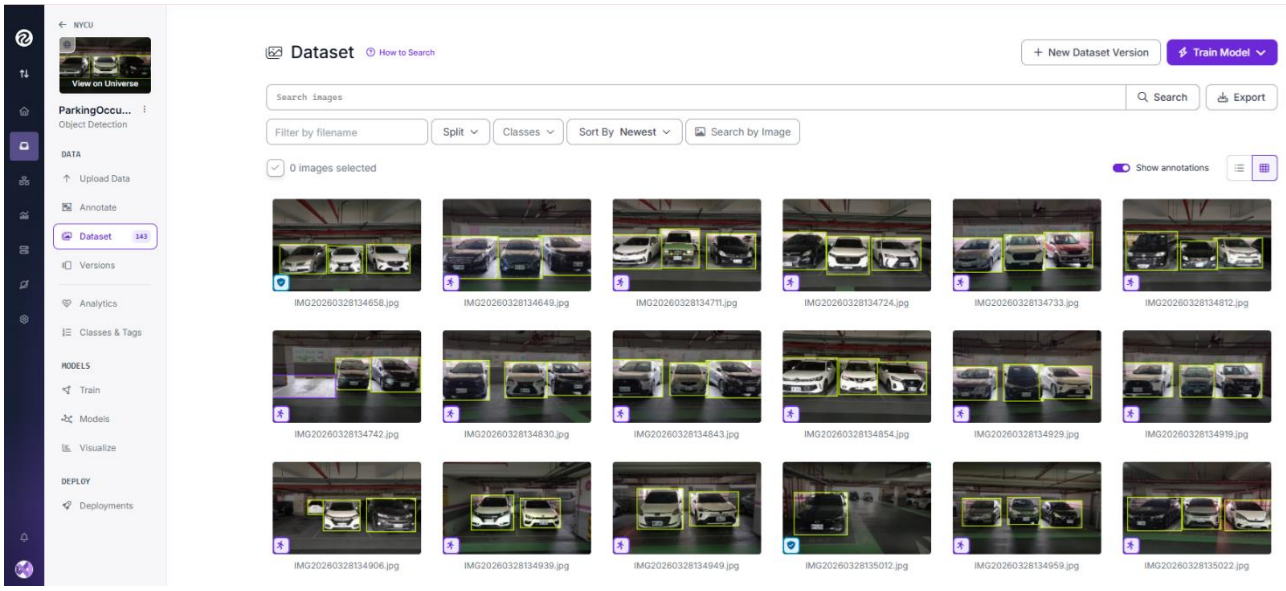
資料蒐集方式如下：

- 使用手機拍攝停車場影像
- 固定攝影機視角
- 擷取不同時間點之停車場畫面
- 確保影像中同時含有 occupied 與 empty 車位
- 影像主要來自同一停車場場景

2.2 Annotation：

使用 Roboflow（https://app.roboflow.com/nycu-7eryh/home）進行標註：每個車位畫 bounding box 標註類別為：

- Empty
- Occupied



【圖 3】Roboflow 標註畫面

完成後匯出 YOLO 格式資料集，包含：

```
dataset/
├── train/
│   ├── images
│   │   ├──train_001.jpg
│   ├── labels
│   │   ├──train_001.txt
├── valid/
│   ├── images
│   │   ├──valid_001.jpg
│   ├── labels
│   │   ├──valid_001.txt
├── test /
│   ├── images
│   │   ├──test_001.jpg
│   ├── labels
│   │   ├──test_001.txt
└── data.yaml
```

2.3 Crop Generation：

- 將 YOLO 標註後下載，透過 Python 程式轉為 crop 小圖
- 每張圖代表一個車位
- resize 為固定大小（128×128）
- 資料集檔案結構：

```
dataset/
├── occupied/
│   ├── car_001.jpg
│   ├── car_002.jpg
│   └── ...
└── empty/
    ├── empty_001.jpg
    ├── empty_002.jpg
    └── ...
```

所有 crop 圖片皆統一 resize 至 128 × 128，以確保 HOG 特徵維度一致。這是因為 HOG 特徵長度與輸入尺寸相關，若尺寸不一致，SVM 將無法直接訓練。

2.4 Dataset Statistics：

本研究最終用於 HOG + SVM 之 crop 資料量如下：

類別	數量
Occupied	471
Empty	387
總數	858

驗證時，YOLO26 顯示驗證集共有：

- Images：87
- Instances：234

其中：

- Empty：129
- Occupied：105

3.5 Dataset Documentation：

依照作業要求，資料集需在報告中說明其：

- amount and composition
- data collection condition
- collection process
- examples

因此本研究資料集之說明如下：

- 資料型態：停車場彩色影像
- 標籤類別：Empty / Occupied
- 資料來源：作者自行拍攝
- 標註工具：Roboflow
- 資料處理：YOLO 標註轉換為單一車位 crop
- 範例：報告中附上 occupied 與 empty 車位小圖

Methods

作業要求至少比較兩種 supervised learning 方法，且至多一種可為 deep learning。
因此本研究選擇：

- HOG + SVM：傳統機器學習方法
- YOLO26：深度學習方法

4.1 Method 1: HOG + SVM：

4.1.1 Overview：

HOG（Histogram of Oriented Gradients）是一種常用的手工特徵，能描述局部區域中的梯度方向分布，對物體形狀與邊緣資訊有良好表現。SVM（Support Vector Machine）則是一種常見的 supervised learning 分類器，適合中小型資料集與高維特徵空間。

4.1.2 Processing Pipeline：

HOG + SVM 流程如下：

- 讀取車位 crop 小圖
- resize 為 128 × 128
- 轉為灰階
- 萃取 HOG 特徵
- 使用 Linear SVM 訓練
- 對單一車位輸出 occupied 或 empty

4.1.3 HOG Parameters：

本研究使用之 HOG 參數為：

- pixels_per_cell = (8, 8)
- cells_per_block = (2, 2)
- block_norm = L2-Hys

在此設定下，每張 128 × 128 影像會產生 8100 維特徵。

4.1.4 SVM Classifier：

本研究使用 LinearSVC 作為分類器。其優點為：

- 適合高維特徵
- 訓練速度快
- 作為傳統 baseline 非常合適

4.2 Method 2: YOLO26：

4.2.1 Overview：

YOLO26 為 Ultralytics 最新版本 YOLO 模型，具備端到端偵測能力。相較於傳統方法，YOLO 可直接對整張影像進行目標定位與分類，較適合應用於實際停車場畫面。

4.2.2 Input and Output：

- 輸入：整張停車場影像

- 輸出：每個標註框之：
  - 類別（Empty / Occupied）
  - Bounding box
  - confidence

4.2.3 Training Setup：

本研究使用 yolo26n.pt 作為初始模型，主要考量 GTX 1650 顯示卡之 4GB VRAM 限制，因此採用較輕量模型進行訓練。

主要訓練參數：

- epochs = 100
- imgsz = 640
- batch = 4
- patience = 20
- pretrained = True

訓練裝置為 GPU，使用 PyTorch CUDA 12.1 環境。

Experimental Setup

5.1 Evaluation Metric：

依據作業要求，classification 應使用 accuracy、confusion matrix 等指標；對二元分類問題，precision、recall、F1-score 亦應納入考量。

因此本研究使用以下指標：

- HOG + SVM
  - Accuracy
  - Precision
  - Recall
  - F1-score
  - Confusion Matrix
- YOLO26
  - Precision
  - Recall
  - mAP@0.5
  - mAP@0.5:0.95

5.2 Cross-Validation：

作業明確要求 supervised learning 需使用 cross-validation。

因此 HOG + SVM 使用 5-fold cross-validation。

5.3 Environment：

本研究之主要實驗環境如下：

- OS：Windows
- Python：3.11.5
- PyTorch：2.5.1 + cu121
- GPU：NVIDIA GeForce GTX 1650
- Ultralytics：8.4.30

Experimental Results

6.1 HOG + SVM Results：

HOG + SVM 訓練輸出如下：

- occupied：471

- empty：387
- total：858
- feature dimension：8100

5-fold cross-validation 結果為：

指標	平均值
Accuracy	1.0000
Precision	1.0000
Recall	1.0000
F1-score	1.0000

Overall Confusion Matrix 為：
[[387 0] [0 471]]
表面上看來，HOG + SVM 在本資料集上達到了完美結果。

6.2 HOG + SVM Prediction Results：
在單一車位 crop 小圖的個別測試中，模型亦能正確輸出：
- occupied → occupied
- empty → empty

此外，在批次預測資料夾中，所選圖片之預測結果也皆正確，說明 HOG + SVM 預測流程已成功實現。

6.3 YOLO26 Results：
YOLO26 在訓練完成後，最佳結果出現在 epoch 80，並由 EarlyStopping 機制於 100 epochs 內提前停止。
驗證結果如下：
Overall

指標	數值
Precision	0.988
Recall	0.973
mAP@0.5	0.994
mAP@0.5:0.95	0.911

Per Class

類別	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Empty	1.000	0.965	0.993	0.899
Occupied	0.976	0.981	0.994	0.924

Discussion

7.1 Why HOG + SVM Achieved 100%：
雖然 HOG + SVM 取得 100% 的 Accuracy、Precision、Recall 與 F1-score，但這樣的結果在真實影像問題中並不常見，因此需要進一步分析。
本研究認為，造成此結果的可能原因包括：
- 固定攝影機與固定車位：
  背景影像來自同一場景，背景高度相似，模型可能學到背景差異，而不只是車位是否有車。
- 資料分割方式可能造成資料洩漏：
  若同一張原始影像中切出的不同車位 crop 同時分散到 train 與 test fold，則模型在測試時實際上已看過相近背景，導致結果過度樂觀。

- 樣本難度較低：
  目前資料主要來自固定視角、白天場景，缺少夜間、強陰影、模糊或跨格停車等困難情況。

因此，雖然 HOG + SVM 在目前資料上表現非常好，但不能直接推論其在更複雜場景中也一定表現同等優秀。

7.2 Comparison Between HOG + SVM and YOLO26：

HOG + SVM 與 YOLO26 各有優缺點。

HOG + SVM：

- 優點：
  - 實作簡單
  - 訓練速度快
  - 對小型、固定場景資料集表現良
- 缺點：
  - 依賴手工特徵
  - 泛化能力有限
  - 對光照、背景與視角變化較敏感

YOLO26

- 優點：
  - 可直接處理整張影像
  - 同時完成定位與分類
  - 更具擴充性與實務應用潛力
- 缺點：
  - 訓練成本較高
  - 需 GPU 加速
  - 對資料量與標註品質更敏感

7.3 On One-Car-Two-Spaces Scenario：

若實際應用中出現一車占用兩格的情況，單純以「總車位數 - 車輛數」計算空位數會產生誤差。因此本研究採用「以車位為單位」的 occupied/empty 標註方式，讓一台跨格停車的車輛可以同時對兩個車位產生 occupied 標籤，這樣較符合實際停車管理需求。

7.4 Practical Insight：

本研究實作過程中發現，若停車場視角固定、車位位置穩定，則 even 傳統方法也能達到不錯表現；但若未來要擴充至：

- 多停車場
- 夜間環境
- 光照變化
- 不同攝影角度
- 遮蔽與跨格停車

則深度學習方法將更具優勢。

What Was Learned

根據作業要求，需說明從實驗中學到什麼，以及仍有哪些問題待解決。

本研究學到的重點包括：

資料集品質比模型選擇更關鍵

若資料來源單一、背景固定，模型結果可能過度樂觀。

- 傳統方法在固定場景下仍具價值：

HOG + SVM 在小型固定場景中表現出高度可行性。

- 深度學習方法更適合未來擴充：
  YOLO26 不僅能分類，還能定位，較適合部署於完整停車場辨識系統。
- 評估設計需避免 data leakage：
  單純 random split 對 crop 資料可能不夠嚴謹，應採用 group-based splitting。

Limitations

本研究限制如下：

- 影像主要來自單一停車場
- 場景與背景變化有限
- 夜間與極端光照資料不足
- HOG + SVM 的 cross-validation 可能受資料洩漏影響
- 未進一步測試不同停車場之泛化能力

Future Work

若有更多時間，本研究希望進一步進行以下工作：

- 增加夜間、雨天、陰影與反光場景
- 以原始影像為單位進行 group K-fold 驗證
- 嘗試 CNN classifier 與 YOLO classification 模式
- 建立即時影片版停車位辨識系統
- 納入一車跨兩格與遮蔽判定規則

Conclusion

本研究依照作業要求，自行建立停車場車位影像資料集，並比較 HOG + SVM 與 YOLO26 兩種 supervised learning 方法。作業要求重點包括：自行建構資料集、至少兩種 supervised learning 方法、使用 cross-validation、進行實驗分析與討論，本研究皆已完整實作。
實驗結果顯示，HOG + SVM 在目前固定場景資料上可達到極高分類表現，而 YOLO26 在整張影像偵測任務上也取得優異的 Precision、Recall 與 mAP。進一步分析指出，固定背景與資料分割方式可能造成 HOG + SVM 過度樂觀的結果，因此對於更真實的實務應用，仍需以更具挑戰性的資料與更嚴謹的驗證方式進一步評估。
整體而言，本研究證明了停車場車位在席辨識可透過傳統與深度學習方法有效完成，且深度學習方法在未來擴充與部署上更具潛力。

References

1. Ultralytics. YOLO Documentation.
2. Dalal, N. and Triggs, B. Histograms of Oriented Gradients for Human Detection.
3. Cortes, C. and Vapnik, V. Support-Vector Networks.
4. OpenCV Documentation.
5. scikit-learn Documentation.
6. Artificial Intelligence Homework 1 Specification.

Appendix A. Program Code

本研究附錄程式包含：

1. Crop.py：由 YOLO 標註框裁切 occupied / empty 小圖
2. hog_svm_train.py：HOG + SVM 訓練程式
3. hog_svm_predict.py：HOG + SVM 單張預測
4. hog_svm_predict_batch.py：HOG + SVM 資料夾批次預測
5. yolo26_train.py：YOLO26 訓練程式
6. yolo26_val.py：YOLO26 驗證程式
7. data.yaml：YOLO26 yaml

Crop.py：由 YOLO 標註框裁切 occupied / empty 小圖
程式設計：劉順松（414551016）

```
import os
import cv2
import yaml
from pathlib import Path

def load_class_names(data_yaml_path):
    """
    從 Roboflow 匯出的 data.yaml 讀取類別名稱
    """
    with open(data_yaml_path, "r", encoding="utf-8") as f:
        data = yaml.safe_load(f)

    names = data.get("names", {})
    if isinstance(names, list):
        class_names = {i: name for i, name in enumerate(names)}
    elif isinstance(names, dict):
        class_names = {int(k): v for k, v in names.items()}
    else:
        raise ValueError("data.yaml 中找不到有效的 names 欄位")

    return class_names


def yolo_to_xyxy(x_center, y_center, width, height, img_w, img_h):
    """
    YOLO normalized bbox -> pixel bbox
    """
    x_center *= img_w
    y_center *= img_h
    width *= img_w
    height *= img_h

    x1 = int(x_center - width / 2)
    y1 = int(y_center - height / 2)
    x2 = int(x_center + width / 2)
    y2 = int(y_center + height / 2)

    return x1, y1, x2, y2


def clamp_bbox(x1, y1, x2, y2, img_w, img_h):
    x1 = max(0, min(x1, img_w - 1))
    y1 = max(0, min(y1, img_h - 1))
    x2 = max(1, min(x2, img_w))
    y2 = max(1, min(y2, img_h))
    return x1, y1, x2, y2


def crop_from_yolo_dataset(
    images_dir,
    labels_dir,
    data_yaml_path,
    output_dir,
    target_classes=("Occupied", "Empty"),
    resize=None,
    save_preview=False
):
```

```
    """
    從 YOLO 格式資料集裁切 occupied / empty 小圖

    參數:
    - images_dir: 圖片資料夾
    - labels_dir: YOLO txt 標註資料夾
    - data_yaml_path: Roboflow 匯出的 data.yaml
    - output_dir: 輸出資料夾
    - target_classes: 想保留的類別
    - resize: tuple，例如 (128, 128)，若不想 resize 則設 None
    - save_preview: 是否另外存帶框預覽圖
    """
    images_dir = Path(images_dir)
    labels_dir = Path(labels_dir)
    output_dir = Path(output_dir)

    class_names = load_class_names(data_yaml_path)

    # 建立輸出資料夾
    for cls in target_classes:
        (output_dir / cls).mkdir(parents=True, exist_ok=True)

    if save_preview:
        (output_dir / "preview").mkdir(parents=True, exist_ok=True)

    image_exts = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
    image_files = [p for p in images_dir.iterdir() if p.suffix.lower() in
image_exts]

    total_saved = 0

    for img_path in image_files:
        label_path = labels_dir / f"{img_path.stem}.txt"
        if not label_path.exists():
            continue

        img = cv2.imread(str(img_path))
        if img is None:
            print(f"[WARN] 無法讀取圖片: {img_path}")
            continue

        img_h, img_w = img.shape[:2]
        preview = img.copy()

        with open(label_path, "r", encoding="utf-8") as f:
            lines = f.readlines()

        obj_count = 0

        for i, line in enumerate(lines):
            parts = line.strip().split()
            if len(parts) != 5:
                continue

            class_id = int(parts[0])
            x_center, y_center, width, height = map(float, parts[1:])

            class_name = class_names.get(class_id, f"class_{class_id}")
            if class_name not in target_classes:
                continue
```

```
            x1, y1, x2, y2 = yolo_to_xyxy(x_center, y_center, width, height,
img_w, img_h)
            x1, y1, x2, y2 = clamp_bbox(x1, y1, x2, y2, img_w, img_h)

            if x2 <= x1 or y2 <= y1:
                continue

            crop = img[y1:y2, x1:x2]
            if crop.size == 0:
                continue

            if resize is not None:
                crop = cv2.resize(crop, resize)

            save_name = f"{img_path.stem}_{i:03d}.jpg"
            save_path = output_dir / class_name / save_name
            cv2.imwrite(str(save_path), crop)

            total_saved += 1
            obj_count += 1

            if save_preview:
                color = (0, 255, 0) if class_name == "Occupied" else (0, 165, 255)
                cv2.rectangle(preview, (x1, y1), (x2, y2), color, 2)
                cv2.putText(
                    preview,
                    class_name,
                    (x1, max(y1 - 8, 20)),
                    cv2.FONT_HERSHEY_SIMPLEX,
                    0.6,
                    color,
                    2,
                    cv2.LINE_AA
                )

        if save_preview and obj_count > 0:
            preview_path = output_dir / "preview" / f"{img_path.stem}_preview.jpg"
            cv2.imwrite(str(preview_path), preview)

    print(f"[DONE] 總共儲存 {total_saved} 張 crop 小圖")
    print(f"[OUTPUT] {output_dir}")


if __name__ == "__main__":
    # ===== 請改成你自己的路徑 =====
    images_dir = "DataAnnotate/ParkingOccupancy.v2i.yolo26_Gray/train/images"
    labels_dir = "DataAnnotate/ParkingOccupancy.v2i.yolo26_Gray/train/labels"
    data_yaml_path = "DataAnnotate/ParkingOccupancy.v2i.yolo26_Gray/data.yaml"
    output_dir = "DataAnnotate/cropped_dataset"
    '''
    images_dir = "DataAnnotate/dataset/train/images"
    labels_dir = "DataAnnotate/dataset/train/labels"
    data_yaml_path = "DataAnnotate/dataset/data.yaml"
    output_dir = "DataAnnotate/cropped_dataset"
    '''

    crop_from_yolo_dataset(
        images_dir=images_dir,
        labels_dir=labels_dir,
```

```
        data_yaml_path=data_yaml_path,
        output_dir=output_dir,
        target_classes=("Occupied", "Empty"),
        resize=(128, 128),      # HOG+SVM 建議固定尺寸
        save_preview=True
    )
```

hog_svm_train.py：HOG + SVM 訓練程式：
程式設計：劉順松（414551016）

```
import os
import cv2
import joblib
import numpy as np
from pathlib import Path
from skimage.feature import hog
from sklearn.svm import LinearSVC
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
)

# ==========================
# 參數設定
# ==========================
DATASET_DIR = "cropped_dataset"      # 資料夾，底下要有 occupied / empty
MODEL_OUTPUT = "hog_svm_model.joblib"

IMG_SIZE = (128, 128)                # 建議固定大小
RANDOM_SEED = 42
N_SPLITS = 5

# HOG 參數
HOG_ORIENTATIONS = 9
HOG_PIXELS_PER_CELL = (8, 8)
HOG_CELLS_PER_BLOCK = (2, 2)
HOG_BLOCK_NORM = "L2-Hys"

# ==========================
# 讀取資料
# ==========================
def load_images_from_folder(folder_path, label):
    """
    從指定資料夾讀取所有圖片，回傳 images, labels, paths
    """
    folder = Path(folder_path)
    image_exts = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

    images = []
    labels = []
    paths = []

    for img_path in folder.iterdir():
        if img_path.suffix.lower() not in image_exts:
            continue

        img = cv2.imread(str(img_path))
        if img is None:
            print(f"[WARN] 無法讀取圖片: {img_path}")
            continue

        images.append(img)
```

```
        labels.append(label)
        paths.append(str(img_path))

    return images, labels, paths


def load_dataset(dataset_dir):
    """
    載入 cropped_dataset/occupied 和 cropped_dataset/empty
    label:
      occupied = 1
      empty    = 0
    """
    dataset_dir = Path(dataset_dir)

    occupied_dir = dataset_dir / "occupied"
    empty_dir = dataset_dir / "empty"

    if not occupied_dir.exists():
        raise FileNotFoundError(f"找不到資料夾: {occupied_dir}")
    if not empty_dir.exists():
        raise FileNotFoundError(f"找不到資料夾: {empty_dir}")

    occ_images, occ_labels, occ_paths = load_images_from_folder(occupied_dir, 1)
    emp_images, emp_labels, emp_paths = load_images_from_folder(empty_dir, 0)

    images = occ_images + emp_images
    labels = occ_labels + emp_labels
    paths = occ_paths + emp_paths

    print(f"[INFO] occupied: {len(occ_images)}")
    print(f"[INFO] empty   : {len(emp_images)}")
    print(f"[INFO] total   : {len(images)}")

    return images, np.array(labels), paths


# ========================
# 特徵處理
# ========================
def preprocess_image(img, img_size=(128, 128)):
    """
    影像前處理：
    1. resize
    2. 轉灰階
    """
    img_resized = cv2.resize(img, img_size)
    gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)
    return gray


def extract_hog_features(gray_img):
    """
    從灰階圖擷取 HOG 特徵
    """
    features = hog(
        gray_img,
        orientations=HOG_ORIENTATIONS,
        pixels_per_cell=HOG_PIXELS_PER_CELL,
        cells_per_block=HOG_CELLS_PER_BLOCK,
```

```
        block_norm=HOG_BLOCK_NORM,
        transform_sqrt=True,
        feature_vector=True
    )
    return features


def build_feature_matrix(images, img_size=(128, 128)):
    """
    將所有影像轉成 HOG 特徵矩陣
    """
    feature_list = []

    for img in images:
        gray = preprocess_image(img, img_size)
        feat = extract_hog_features(gray)
        feature_list.append(feat)

    X = np.array(feature_list)
    return X


# ========================
# 交叉驗證
# ========================
def run_cross_validation(X, y, n_splits=5, random_seed=42):
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
random_state=random_seed)

    acc_scores = []
    precision_scores = []
    recall_scores = []
    f1_scores = []

    all_y_true = []
    all_y_pred = []

    for fold, (train_idx, test_idx) in enumerate(skf.split(X, y), start=1):
        print(f"\n========== Fold {fold} ==========")

        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        model = LinearSVC(
            C=1.0,
            max_iter=10000,
            random_state=random_seed
        )

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, zero_division=0)
        recall = recall_score(y_test, y_pred, zero_division=0)
        f1 = f1_score(y_test, y_pred, zero_division=0)
        cm = confusion_matrix(y_test, y_pred)

        acc_scores.append(acc)
        precision_scores.append(precision)
```

```
        recall_scores.append(recall)
        f1_scores.append(f1)

        all_y_true.extend(y_test.tolist())
        all_y_pred.extend(y_pred.tolist())

        print(f"Accuracy : {acc:.4f}")
        print(f"Precision: {precision:.4f}")
        print(f"Recall   : {recall:.4f}")
        print(f"F1-score : {f1:.4f}")
        print("Confusion Matrix:")
        print(cm)
        print("Classification Report:")
        print(classification_report(
            y_test,
            y_pred,
            target_names=["empty", "occupied"],
            digits=4,
            zero_division=0
        ))

    print("\n========== Cross-validation Summary ==========")
    print(f"Mean Accuracy : {np.mean(acc_scores):.4f} ±
{np.std(acc_scores):.4f}")
    print(f"Mean Precision: {np.mean(precision_scores):.4f} ±
{np.std(precision_scores):.4f}")
    print(f"Mean Recall   : {np.mean(recall_scores):.4f} ±
{np.std(recall_scores):.4f}")
    print(f"Mean F1-score : {np.mean(f1_scores):.4f} ± {np.std(f1_scores):.4f}")

    overall_cm = confusion_matrix(all_y_true, all_y_pred)
    print("\nOverall Confusion Matrix (all folds combined):")
    print(overall_cm)

    print("\nOverall Classification Report:")
    print(classification_report(
        all_y_true,
        all_y_pred,
        target_names=["empty", "occupied"],
        digits=4,
        zero_division=0
    ))

    results = {
        "mean_accuracy": np.mean(acc_scores),
        "std_accuracy": np.std(acc_scores),
        "mean_precision": np.mean(precision_scores),
        "std_precision": np.std(precision_scores),
        "mean_recall": np.mean(recall_scores),
        "std_recall": np.std(recall_scores),
        "mean_f1": np.mean(f1_scores),
        "std_f1": np.std(f1_scores),
        "overall_confusion_matrix": overall_cm,
    }

    return results


# ========================
# 訓練最終模型並存檔
```

```
# ==========================
def train_final_model(X, y, output_path):
    model = LinearSVC(
        C=1.0,
        max_iter=10000,
        random_state=RANDOM_SEED
    )
    model.fit(X, y)

    payload = {
        "model": model,
        "img_size": IMG_SIZE,
        "class_names": {0: "empty", 1: "occupied"},
        "hog_params": {
            "orientations": HOG_ORIENTATIONS,
            "pixels_per_cell": HOG_PIXELS_PER_CELL,
            "cells_per_block": HOG_CELLS_PER_BLOCK,
            "block_norm": HOG_BLOCK_NORM,
        }
    }

    joblib.dump(payload, output_path)
    print(f"\n[INFO] 最終模型已儲存: {output_path}")


# ==========================
# 主程式
# ==========================
def main():
    print("Loading dataset...")
    images, labels, paths = load_dataset(DATASET_DIR)

    if len(images) == 0:
        raise RuntimeError("資料集沒有任何圖片，請檢查 cropped_dataset/occupied 和
cropped_dataset/empty")

    print("\nExtracting HOG features...")
    X = build_feature_matrix(images, IMG_SIZE)
    y = labels

    print(f"[INFO] Feature matrix shape: {X.shape}")
    print(f"[INFO] Labels shape        : {y.shape}")

    print("\nRunning 5-fold cross-validation...")
    results = run_cross_validation(X, y, n_splits=N_SPLITS,
random_seed=RANDOM_SEED)

    print("\nTraining final model on all data...")
    train_final_model(X, y, MODEL_OUTPUT)

    print("\n[DONE] HOG + SVM training completed.")


if __name__ == "__main__":
    main()
```

hog_svm_predict.py：HOG + SVM 單張預測
程式設計：劉順松（414551016）

```
import cv2
import joblib
from skimage.feature import hog


def preprocess_image(img, img_size):
    img_resized = cv2.resize(img, img_size)
    gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)
    return gray


def extract_hog_features(gray_img, hog_params):
    feat = hog(
        gray_img,
        orientations=hog_params["orientations"],
        pixels_per_cell=tuple(hog_params["pixels_per_cell"]),
        cells_per_block=tuple(hog_params["cells_per_block"]),
        block_norm=hog_params["block_norm"],
        transform_sqrt=True,
        feature_vector=True
    )
    return feat


def predict_image(image_path, model_path="hog_svm_model.joblib"):
    payload = joblib.load(model_path)

    model = payload["model"]
    img_size = tuple(payload["img_size"])
    class_names = payload["class_names"]
    hog_params = payload["hog_params"]

    img = cv2.imread(image_path)
    if img is None:
        raise FileNotFoundError(f"Cannot read image: {image_path}")

    gray = preprocess_image(img, img_size)
    feat = extract_hog_features(gray, hog_params).reshape(1, -1)

    pred = model.predict(feat)[0]
    print(f"Prediction: {class_names[pred]}")


if __name__ == "__main__":
    #test_image = "IMG20260328134711_Occupied.jpg"   # 測試圖
    test_image = "IMG20260328140416_Empty.jpg"   # 測試圖
    predict_image(test_image)
```

hog_svm_predict_batch.py：HOG + SVM 資料夾批次預測
程式設計：劉順松（414551016）

```
import cv2
import joblib
from pathlib import Path
from skimage.feature import hog


def preprocess_image(img, img_size):
    """
    將圖片 resize 並轉為灰階
    """
    img_resized = cv2.resize(img, img_size)
    gray = cv2.cvtColor(img_resized, cv2.COLOR_BGR2GRAY)
    return gray


def extract_hog_features(gray_img, hog_params):
    """
    擷取 HOG 特徵
    """
    feat = hog(
        gray_img,
        orientations=hog_params["orientations"],
        pixels_per_cell=tuple(hog_params["pixels_per_cell"]),
        cells_per_block=tuple(hog_params["cells_per_block"]),
        block_norm=hog_params["block_norm"],
        transform_sqrt=True,
        feature_vector=True
    )
    return feat


def predict_folder(folder_path, model_path="hog_svm_model.joblib"):
    """
    預測資料夾內全部圖片
    """
    payload = joblib.load(model_path)

    model = payload["model"]
    img_size = tuple(payload["img_size"])
    class_names = payload["class_names"]
    hog_params = payload["hog_params"]

    folder = Path(folder_path)
    if not folder.exists():
        raise FileNotFoundError(f"找不到資料夾: {folder_path}")

    image_exts = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
    image_files = sorted([p for p in folder.iterdir() if p.suffix.lower() in
image_exts])

    if not image_files:
        print(f"[WARN] 資料夾內沒有圖片: {folder_path}")
        return

    for img_path in image_files:
        img = cv2.imread(str(img_path))
        if img is None:
```

```
            print(f"[WARN] 無法讀取圖片: {img_path.name}")
            continue

        gray = preprocess_image(img, img_size)
        feat = extract_hog_features(gray, hog_params).reshape(1, -1)

        pred = model.predict(feat)[0]
        label = class_names[pred]

        print(f"Prediction: {img_path.name} + {label}")


if __name__ == "__main__":
    # 測試圖片資料夾
    test_folder = "test_images"
    predict_folder(test_folder)
```

yolo26_train.py：YOLO26 訓練程式
程式設計：劉順松（414551016）

```
from ultralytics import YOLO
import torch


def main():
    # 自動判斷是否可使用 GPU
    # 如果 torch.cuda.is_available() 為 True，則使用第 0 張 GPU
    # 否則改用 CPU
    device = 0 if torch.cuda.is_available() else "cpu"
    print("Using device:", device)

    # 載入 YOLO26 預訓練模型
    # 可依需求改成 yolo26s.pt、yolo26m.pt 等
    # GTX 1650 建議先用 yolo26n.pt
    model = YOLO("yolo26n.pt")

    # 開始訓練
    results = model.train(
        # data.yaml 路徑
        data=r"C:\Users\LSS\Documents\114 下\AI\AI-Hw1\Yolo26\dataset\data.yaml",

        # 訓練 epoch 數
        epochs=100,

        # 輸入影像大小
        imgsz=640,

        # batch size
        # GTX 1650 4GB 建議先用 4，若顯存不足可改 2
        batch=4,

        # 訓練裝置
        device=device,

        # DataLoader worker 數
        # Windows 建議先保守設 2，若不穩可設 0
        workers=2,

        # 訓練結果輸出資料夾
        project=r"C:\Users\LSS\Documents\114 下\AI\AI-Hw1\Yolo26\runs",

        # 本次實驗名稱
        name="parking_occupancy_yolo26",

        # 是否使用預訓練權重
        pretrained=True,

        # Early stopping 的 patience
        # 若連續 20 個 epoch 沒有改善，就停止訓練
        patience=20,

        # 儲存模型
        save=True,

        # 繪製訓練曲線與結果圖
        plots=True
    )
```

```
    # 顯示訓練完成訊息
    print("Training finished.")

    # 印出訓練回傳結果物件
    print(results)

    # 這裡不再額外呼叫 model.val()
    # 原因：
    # 訓練過程中 YOLO 已經自動做 validation
    # 你前面遇到的 Invalid device id 就是出現在 train 後再手動 val()
    # 所以建議把驗證獨立成另一支程式 yolo26_val.py


if __name__ == "__main__":
    main()
```

yolo26_val.py：YOLO26 驗證程式
程式設計：劉順松（414551016）

```
from ultralytics import YOLO
import torch


def main():
    # 自動判斷是否可用 GPU
    device = 0 if torch.cuda.is_available() else "cpu"
    print("Using device:", device)

    # 載入訓練完成後的最佳模型
    model = YOLO(
        r"C:\Users\LSS\Documents\114下\AI\AI-
Hw1\Yolo26\runs\parking_occupancy_yolo262\weights\best.pt"
    )

    # 執行驗證
    metrics = model.val(
        # 指定 data.yaml
        data=r"C:\Users\LSS\Documents\114下\AI\AI-Hw1\Yolo26\dataset\data.yaml",

        # 指定裝置
        device=device,

        # 驗證影像大小
        imgsz=640,

        # 指定要驗證的資料切分
        # 可選 "val" 或 "test"
        split="val"
    )

    # 印出驗證結果
    print("Validation finished.")
    print(metrics)


if __name__ == "__main__":
    main()
```

data.yaml：YOLO26 yaml
程式設計：劉順松（414551016）

```
path: C:/Users/LSS/Documents/114下/AI/AI-Hw1/Yolo26/dataset
train: train/images
val: valid/images
test: test/images

nc: 2
names: ['Empty', 'Occupied']
```